Dept. of Computer Science and Engineering, Czech Technical University in Prague
Faculty of Mathematics and Physics, Charles University in Prague
Faculty of Electrical Engineering and Computer Science, Technical University of Ostrava
Faculty of Informatics and Information Technologies, Slovak University of Technology
Faculty of Informatics, Masaryk University
Faculty of Management Science and Informatics, University of Zilina

CTU Open Contest 2008

Save the World!

Dear Contestants!

Global world-wide bank and financial crisis is here. Did you notice? Will you help to overcome it and hence save the world?

Your task is to create computer software to solve several problems related to financial, governmental, or banking environment. You may use C, C++, Pascal, or Java programming languages. The choice is yours but you will be fully responsible for the speed of your solution.

All of your programs will read one single file from the standard input. The results will be written to the standard output. You are not allowed to use any other files. Input and output formats are described in problem statements and must be exactly obeyed. If not specified otherwise, all input numbers and results will fit into a signed 32-bit integer type.

As mentioned before, we need the correct answer produced in some appropriate time. Nothing else matters. You may choose any algorithm, any programming style.

Good luck in the Czech Technical University Open Contest 2008. We hope you will enjoy it!

Your organizing team

Aquasoft Informační systémy

Sun microsystems

ET NETERA

This problem set consists of ten sheets of paper (including this one) and contains nine problems. Please make sure that you have the complete set.

Dept. of Computer Science and Engineering, Czech Technical University in Prague
Faculty of Mathematics and Physics, Charles University in Prague
Faculty of Electrical Engineering and Computer Science, Technical University of Ostrava
Faculty of Informatics and Information Technologies, Slovak University of Technology
Faculty of Informatics, Masaryk University
Faculty of Management Science and Informatics, University of Zilina

CTU Open Contest 2008

Alea iacta est

`alea.c`, `alea.C`, `alea.java`, `alea.p`

Isaac B. Manfred always dreamed about being a terribly rich man. When he was a child, he played with banknotes instead of toys. After finishing high school, he quickly became a senior broker in one famous bank. His career rose rapidly and also did his wealth. Unfortunately, the bank crisis changed his life significantly. The broker suddenly became broke.

To gain his money back, I.B.M. spent a lot of time in casinos, trying to earn some money there. Most people who ever tried to get rich in casinos are actually very poor today. But this does not include I.B.M. He is a very clever guy and instead of blindly betting his money, he carefully studies various games and computes the probabilities of losing or winning. First, he tried his luck with Roulette and Blackjack, but found out that the odds of winning a fortune are low.

Recently, I.B.M. started to study dice games. He found several of them similar to a trademarked game called Yahtzee! The rules sometimes vary but basic principles are the same. To give you an idea, we will describe a simplified version of such rules.

The game consists of rounds. In each round, a player rolls five dice. After the first roll, it is possible to keep some of the dice and re-roll the rest of them. Any number of dice can be re-rolled (including none or all of them). If the re-rolled dice still do not fit the player's intentions, it is possible to re-roll some of them again, for the third and final time. After at most two such re-rolls, the player must assign the result to one of possible combinations and the round is scored according to that combination.

The following table shows the list of combinations, conditions that must be satisfied to use them, and the number of points scored when the combination is used.

Combination	**Condition**	**Scoring**	**Example**	**Score**
Ones	At least one 1.	One point for each 1.	1 1 2 4 5	2
Twos	At least one 2.	Two points for each 2.	1 2 2 2 6	6
Threes	At least one 3.	Three points for each 3.	1 2 4 5 5	0
Fours	At least one 4.	Four points for each 4.	4 4 4 4 4	20
Fives	At least one 5.	Five points for each 5.	1 2 3 4 5	5
Sixes	At least one 6.	Six points for each 6.	1 4 6 6 6	18
Sequence	(1 2 3 4 5) or (2 3 4 5 6).	Thirty points.	1 2 3 4 5	30
Full House	Three of the same value and a pair of another value.	Sum of all dice values.	2 2 5 5 5	19
Four of a kind	Four of the same value, the fifth one different.	Sum of all dice values.	4 4 4 4 5	21
Five of a kind	All five of the same value.	Fifty points.	1 1 1 1 1	50
Chance	None.	Sum of all dice values.	2 4 5 5 6	22

A small example: The player rolls 2, 3, 6, 6, 5. The two 6's are kept and the three remaining dice re-rolled, they give new values: 1, 1, 6. The player may now choose to score 20 points immediately for a Full House. Instead, he or she decides to re-roll the two 1's again, in hope there will be another 6. The dice give 4 and 5 and the player will score either 18 points for Sixes or 27 points for Chance.

The main point of the game is that there are eleven combinations and eleven rounds. During the whole game, each combination must be used exactly once. It may happen that some result would not fit into any available combination. In such a case, the player must select some combination anyway, scoring zero points for that round and losing the possibility to use that combination later. These rules make the game very tricky, especially at the end, when the combinations have been almost exhausted.

Now, we get back to Isaac. He found a casino with an electronic version of this dice game. After carefully watching many games of other players, he was able to crack the random-number generator used in the machine. Therefore, he is able to predict the following rolls exactly. What an opportunity! However, it is still not easy to find the optimal strategy. If you write a program that would help him to become rich, he may share some of his money with you.

Input Specification

The input contains several scenarios, each of them specified on a single line. The line contains three numbers separated by a space: A, C, and X_0. These numbers describe the random-number generator: A is called a *multiplier* ($1 \leq A \leq 2^{31}$), C is an *increment* ($0 \leq C \leq 2^{31}$), and X_0 is the initial *seed* ($0 \leq X_0 \leq 2^{31}$). The last scenario is followed by a line containing three zeros.

The generator is a linear congruential generator, which means that the next random number is calculated from the previous one using the following formula:

$$X_{n+1} = (A * X_n + C) \bmod 2^{32}$$

The modulo operation specifies that only the lowest 32 bits of the result are used, the rest is discarded. Numbers X_1, X_2, X_3 ... constitute a pseudo-random sequence, each of them determines the result of one individual roll of a dice. With congruential generators, the "randomness" of the numbers is in their higher bits only — therefore, to get a result of the n-th roll (starting with $n = 1$), we discard lower 16 bits of the number X_n and compute the remainder when the number in bits 16-31 is divided by six. This gives a number between 0 and 5, by adding one, we get a number shown on a dice:

$$roll(n) = [floor(X_n/2^{16}) \bmod 6] + 1$$

For example, when $A = 69069$, $C = 5$, and the $X_0 = 0$ is zero, we get the following sequence of "random" rolls: 1, 6, 6, 3, 2, 4, 3, 2, 3, 5, 1, 6, 6, 4, 5, 1, 3, 4, 1, ...

Output Specification

For each scenario, print one integer number: the maximal number of points that may be scored in a game determined by the given generator. The score is calculated after 11 rounds as the sum of scores in all combinations.

Sample Input	Output for Sample Input
69069 5 0	235
69069 5 2	194
1664525 1013904223 177	241
1103515245 12345 67890	235
0 0 0	

[1] dice = kostka, kostky, po slovensky kocka ;)

Dept. of Computer Science and Engineering, Czech Technical University in Prague
Faculty of Mathematics and Physics, Charles University in Prague
Faculty of Electrical Engineering and Computer Science, Technical University of Ostrava
Faculty of Informatics and Information Technologies, Slovak University of Technology
Faculty of Informatics, Masaryk University
Faculty of Management Science and Informatics, University of Zilina

CTU Open Contest 2008

On-Line Banking

`banking.c`, `banking.C`, `banking.java`, `banking.p`

A central bank is an institution that is responsible for the monetary policy of a country. Beside others, it monitors and supervises other banks. Your task is to write a computer program that will serve as a simple supervision instrument for monitoring bank accounts.

Input Specification

The input contains several scenarios. Each scenario begins with a line containing a single positive integer A, the number of bank accounts at the beginning of the supervision, $0 < A \leq 100$. Then there are A lines each describing one account. Such a line contains the account number, one space, and a non-negative decimal number specifying the account balance.

Each of the following lines specifies a request for one bank operation. The line begins with a command and then, separated by a space, there are optional parameters. The list of commands follows, the third column in the table shows the number of parameters.

Command	Meaning	Pars	Parameters Description
`create`	Create a new account	1	New account number
`deposit`	Deposit cash to some account	2	Account number, Amount to deposit
`withdraw`	Withdraw cash from an account	2	Account number, Amount to withdraw
`transfer`	Wire transfer of money	3	Source account, Target account, Amount

The number of requests in every scenario will always be between 0 and 1000 (inclusive). The last request of a scenario is followed by the word "`end`" and one empty line. Then the next scenario begins. The last scenario is followed by a line containing zero in place of A.

All account numbers consist of exactly four decimal digits followed by a slash character ("`/`") and one digit specifying a code of the bank that operates the account. Each bank has its own unique code.

Amounts are always given as non-negative decimal numbers with exactly two digits after the decimal point. No amount in the input will be higher than 10 000. No unnecessary leading zeros are permitted, i.e., only amounts strictly less than 1.00 may start with a zero.

Output Specification

For each request, output exactly one line. The line must begin with the command followed by the amount parameter (if present) separated from the command by a space. Then there is a colon ("`:`"), one space and the result of the operation.

create: If there is already an account with the same number in the same bank, the result will be "`already exists`". Otherwise, create the account (with the initial balance of zero) and output "`ok`".

For all commands other than *create*, if there is no account with the number specified as a parameter, the result will always be "`no such account`". If the account exists (or both accounts do), the following rules apply.

deposit: The result is always "`ok`" for existing accounts. Simply add the amount to the account balance.

withdraw: If the balance of the specified account is strictly lower than the requested amount, the result will be "`insufficient funds`". Otherwise, subtract the amount and output "`ok`".

transfer: First, if both account numbers are equal (including the bank code), the result should be "`same account`". Similar to the previous case, if the balance of the source account is lower than the amount to be transferred, the result will be "`insufficient funds`". Otherwise, transfer the money from the source account to the target one and output either "`ok`" if the transfer happens inside one bank or "`interbank`" if the money have to be transferred from one bank to another.

After each scenario, output the word "`end`" and one empty line. After the "`end`" of the last scenario, output one additional line containing the word "`goodbye`".

Sample Input

```
3
1234/5 100.00
4321/6 150.20
5432/5 1600.00
withdraw 1234/5 20.00
deposit 1234/5 35.00
withdraw 1234/5 200.00
transfer 5432/5 1234/5 100.50
transfer 5432/5 4321/6 50.00
create 1234/5
create 1236/5
transfer 1236/5 1236/5 100.00
transfer 1236/5 1234/5 100.00
withdraw 0000/0 100.00
deposit 0000/0 0.11
transfer 1234/5 0000/0 10000.00
end

1
9999/9 9.40
deposit 9999/9 6.92
withdraw 9999/9 9.68
withdraw 9999/9 6.64
end

0
```

Output for Sample Input

```
withdraw 20.00: ok
deposit 35.00: ok
withdraw 200.00: insufficient funds
transfer 100.50: ok
transfer 50.00: interbank
create: already exists
create: ok
transfer 100.00: same account
transfer 100.00: insufficient funds
withdraw 100.00: no such account
deposit 0.11: no such account
transfer 10000.00: no such account
end

deposit 6.92: ok
withdraw 9.68: ok
withdraw 6.64: ok
end

goodbye
```

Dept. of Computer Science and Engineering, Czech Technical University in Prague
Faculty of Mathematics and Physics, Charles University in Prague
Faculty of Electrical Engineering and Computer Science, Technical University of Ostrava
Faculty of Informatics and Information Technologies, Slovak University of Technology
Faculty of Informatics, Masaryk University
Faculty of Management Science and Informatics, University of Zilina

International Collegiate Programming Contest

`contest.c`, `contest.C`, `contest.java`, `contest.p`

Do you have any idea how much work must be done to make a competition like this to happen? Reserve rooms, install computers, buy some food, print certificates — these are only some of the important tasks. Can you imagine how many sleepless nights have been spent since the first CTU competition in 1995? And this was only for people like you to enjoy the ICPC. Wow!

Beside others, the problem set must be carefully prepared. This includes not only the problem statements but also solutions and test data. It is really a large amount of work. Would you help us, please? Your task is to create a test input data for the Problem B (`banking`).

Input Specification

The input to your program will be some valid and correct output for Problem B.

Output Specification

Print any input that will exactly satisfy the Input Specification for the Problem B and that would generate the given output.

Sample Input

```
withdraw 20.00: ok
deposit 35.00: ok
withdraw 200.00: insufficient funds
transfer 100.50: ok
transfer 50.00: interbank
create: already exists
create: ok
transfer 100.00: same account
transfer 100.00: insufficient funds
withdraw 100.00: no such account
deposit 0.11: no such account
transfer 10000.00: no such account
end

deposit 6.92: ok
withdraw 9.68: ok
withdraw 6.64: ok
end

goodbye
```

Output for Sample Input

```
3
1234/5 100.00
4321/6 150.20
5432/5 1600.00
withdraw 1234/5 20.00
deposit 1234/5 35.00
withdraw 1234/5 200.00
transfer 5432/5 1234/5 100.50
transfer 5432/5 4321/6 50.00
create 1234/5
create 1236/5
transfer 1236/5 1236/5 100.00
transfer 1236/5 1234/5 100.00
withdraw 0000/0 100.00
deposit 0000/0 0.11
transfer 1234/5 0000/0 10000.00
end

1
9999/9 9.40
deposit 9999/9 6.92
withdraw 9999/9 9.68
withdraw 9999/9 6.64
end

0
```

In this problem, "Presentation Error" means that your output format is incorrect and the output does not satisfy the Input Specification for Problem B. "Wrong Answer" will be used if your output seems as a correct input for Problem B but it would not generate the requested answer.

Dept. of Computer Science and Engineering, Czech Technical University in Prague
Faculty of Mathematics and Physics, Charles University in Prague
Faculty of Electrical Engineering and Computer Science, Technical University of Ostrava
Faculty of Informatics and Information Technologies, Slovak University of Technology
Faculty of Informatics, Masaryk University
Faculty of Management Science and Informatics, University of Zilina

CTU Open Contest 2008

Careful Declaration

`declare.c`, `declare.C`, `declare.java`, `declare.p`

"Do not panic!" That is not only the inscription on a famous book cover, but it is also a message that all banks need to deliver to their clients during every and each crisis, whatever negligible. If the clients started to panic, it would mean the end for any bank.

To calm down their clients, the banks and governments may issue various declarations stating that there is *really* nothing to worry about. It is very important for such declarations to be consistent, otherwise the people may not believe them.

Imagine that we have two declarations, one suggested by a bank and the other coming from the government. They finally decided to merge those two declarations into a single one and issue it together. (People may think there is some problem if there are too many declarations.) Both the bank and the government insist that the common declaration must contain the whole text of their original declarations.

Your task is to create the joint declaration that will be issued at a press conference tonight. The main condition is that the resulting text must contain all words from the two original proposals, in the same order. Also, we have to keep the declaration as short as possible.

Input Specification

The input contains several test cases, each consisting of two lines of text giving two original declarations. Both lines contain at least one and at most two thousand words separated by a space. Each of the words is formed by lowercase letters of the English alphabet (a–z). Each word will contain at least one such letter and no more than 10 letters. There is a single dot ("`.`") after the last word of each declaration, the dot is also preceded by one space.

There is one additional line with a single dot after the last test case in the input.

Output Specification

Your program has to output a single line of text for each test case. The line should contain a sequence of words separated by a space. The sequence must have the following properties:

1. Both input texts are subsequences of the output text, i.e., all words of each original text appear in the result in the same order, though they may be interleaved by other words.
2. Among all such possible texts, your output must contain the smallest possible number of words.
3. If there are several possibilities of the same length, choose the one that is *lexicographically smallest*.

Output a space and a single dot "`.`" after the last word of the declaration.

Text A is called to be lexicographically smaller than text B, if it "would be first in a regular dictionary". More exactly, if the first word that is different in both texts is lexicographically smaller in text A than in text B.

Similarly, word C is called to be lexicographically smaller then word D, if (a) for the first letter that is different in both words, the letter in word C comes earlier in the alphabet than the letter in word D, or (b) word C is shorter and it is a prefix of word D.

Sample Input

```
all our banks are sound and safe .
all deposits in our bank are safe do not worry .
everything fine firmly under control .
deposits in bank abc are fine like nothing is .
.
```

Output for Sample Input

```
all deposits in our bank banks are sound and safe do not worry .
deposits everything in bank abc are fine firmly like nothing is under control .
```

Dept. of Computer Science and Engineering, Czech Technical University in Prague
Faculty of Mathematics and Physics, Charles University in Prague
Faculty of Electrical Engineering and Computer Science, Technical University of Ostrava
Faculty of Informatics and Information Technologies, Slovak University of Technology
Faculty of Informatics, Masaryk University
Faculty of Management Science and Informatics, University of Zilina

CTU Open Contest 2008

Stock Exchange

`exchange.c`, `exchange.C`, `exchange.java`, `exchange.p`

All stock exchanges in the world have been hit very hardly by the crisis. To keep their profit, they now try to cut down all expenses. Prague Stock Exchange would like to employ new computer software to run their operations.

Will you help them to develop such a system? Your task is to detect all pairs of buyers and sellers that are able to make a deal together.

Input Specification

The input contains a description of several issuers.[1] Each of them begins with a line containing an integer N, one space, and a code of the stock issuer. After the first one, there are N other lines, each representing one bid. A bid consists of an agent name, one space character, a word specifying the bid type in lowercase letters (either "`buy`" or "`sell`") another space character, and a price given as a decimal number with exactly three digits after the decimal point. All *buy* bids always specify the maximal price, *sell* bids list the minimal price for which the agent wants to sell the stocks.

Every issuer code consist of 1–10 uppercase letters ("`A`"–"`Z`"). Agent names have at least 1 and at most 20 characters and may be composed of both lowercase and uppercase letters. For one issuer, all agent names are always unique, but the same agents may post bids for several issuers. The number of bids (N) will never exceed 1000. No price will be higher than 10 000.

The last line of the input contains the string "`0 END`".

Output Specification

For each issuer, print its code on a separate line. Then, for each agent's bid (in the same order they were given in the input), output the name of the agent, a colon ("`:`"), one space, and a list of all agents that are potential counter-parties for the issuer in concern, i.e., one of them wants to buy, the other to sell and the selling price is less or equal to the buying price.

The agent names must be separated by a space and listed in the same order that was used in the input for that issuer. If there is no matching bid, output the string "`NO-ONE`" instead.

[1] issuer = vydavatel akcií, zde ve významu jednoho konkrétního titulu akcie

Sample Input

```
3 IBM
OneBuyer buy 10.600
TooExpensive sell 12.000
ThisWillWork sell 10.600
4 ACM
one sell 129.999
two buy 130.000
three buy 131.000
four sell 129.888
4 CVUT
seller sell 121.110
toopoor buy 121.109
sellertwo sell 121.111
iamok buy 121.112
0 END
```

Output for Sample Input

```
IBM
OneBuyer: ThisWillWork
TooExpensive: NO-ONE
ThisWillWork: OneBuyer
ACM
one: two three
two: one four
three: one four
four: two three
CVUT
seller: iamok
toopoor: NO-ONE
sellertwo: iamok
iamok: seller sellertwo
```

Dept. of Computer Science and Engineering, Czech Technical University in Prague
Faculty of Mathematics and Physics, Charles University in Prague
Faculty of Electrical Engineering and Computer Science, Technical University of Ostrava
Faculty of Informatics and Information Technologies, Slovak University of Technology
Faculty of Informatics, Masaryk University
Faculty of Management Science and Informatics, University of Zilina

CTU Open Contest 2008

Wooden Fence

`fence.c`, `fence.C`, `fence.java`, `fence.p`

Did you ever wonder what happens to your money when you deposit them to a bank account? All banks hold such deposits in various assets, such as gold, stocks, obligations, deposits in other banks, loans, bonds, and many others. Due to the financial crisis and instability of the stock exchanges, many banks find out that stocks are not very reliable and their possession may be too risky.

Therefore, the banks now prefer other assets, especially gold. The main trouble with gold is that there is only a limited amount of it in the whole world. And it is not enough to cover all money held by all banks. (Wait, isn't *this* the real reason of the crisis?)

If there is not enough gold, other commodities must be exploited instead. The International Bank of Monetania (IBM) has recently come up with an idea of using very old and valuable trees as their assets. They bought a piece of land with several such trees and now expect their value to grow. Literally, of course.

Unfortunately, the trees are threatened by wildlife, because animals do not understand their value and nibble them. Moreover, there is a permanent danger of theft. As a result, it is absolutely necessary to build a good solid fence around the trees.

The IBM quickly realized that the only suitable material available to build the fence is the wood from the trees themselves. In other words, it is necessary to cut down some trees in order to build a fence around the remaining ones. Of course, to keep the maximum value, we want to minimize the value of the trees that had to be cut. You are to write a program that solves this problem.

Input Specification

The input contains several test cases, each of which describes one piece of land. Each test case begins with a line containing a single integer N, $2 \leq N \leq 16$, the total number of trees. Each of the subsequent N lines contains 4 integers X_i, Y_i, V_i, L_i separated by at least one space.

The four numbers describe a single tree. (X_i, Y_i) is the position of the tree in the plane, V_i is its value, and L_i is the length of fence that can be built using the wood of the tree. You may assume that $0 \leq V_i, L_i \leq 10\,000$ and $-10\,000 \leq X_i, Y_i \leq 10\,000$. No two trees in a test case will grow at the same position.

The input ends with a line containing zero in place of N.

Output Specification

For each test case, compute a subset of the trees such that, using the wood from that subset, the remaining trees can be enclosed in a single continuous fence. Find the subset with the minimal total value. For simplicity, regard the trees as having zero diameter.

Output one line with the sentence "`The lost value is` T`.`", where T is the minimal value of the trees that must be cut.

Sample Input

```
6
0  0  8  3
1  4  3  2
2  1  7  1
4  1  2  3
3  5  4  6
2  3  9  8
3
3  0 10  3
5 -3 20 25
7 -3 30 32
2
100 0 5 4
0 100 4 5
5
0 0 10 10
0 1 10 10
1 0 10 10
1 1 10 10
50 50 8 4
0
```

Output for Sample Input

```
The lost value is 9.
The lost value is 20.
The lost value is 4.
The lost value is 8.
```

Dept. of Computer Science and Engineering, Czech Technical University in Prague
Faculty of Mathematics and Physics, Charles University in Prague
Faculty of Electrical Engineering and Computer Science, Technical University of Ostrava
Faculty of Informatics and Information Technologies, Slovak University of Technology
Faculty of Informatics, Masaryk University
Faculty of Management Science and Informatics, University of Zilina

CTU Open Contest 2008

Safe Gambling

`gambling.c`, `gambling.C`, `gambling.java`, `gambling.p`

All banks and other financial institutions want their money to earn other money. It is really simple: they invest in some assets (such as stocks), wait until their value grows, and then sell them with a high profit.

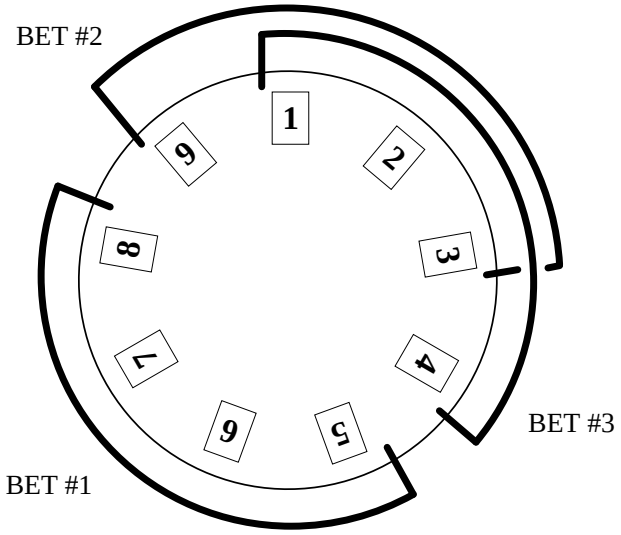
Due to the crisis, the stock exchange rates change too quickly, which caused the stocks to become too risky assets. Since large and creditable banks cannot afford so much risk, they have to search for other activities that bring similar profit but that are more predictable. Such as betting the money in casinos.

You were asked by a manager of one important bank (whose name has to be kept secret, since this is a highly confidential matter) to develop a computer program that would help them to "invest" their money into a new modern roulette, operated by Casino Väter Und Töchter (CVUT).

The roulette is an unusual one. Each pocket ("number") may have a different price that must be paid when that particular number is used in a bet. Moreover, any bet in this roulette must simultaneously cover only adjacent pockets. The price of such a bet is the sum of prices for individual pockets.

Bank managers made a decision that in every bet, one half of the pockets must be covered. They supposed that if two such bets are made, all pockets will be covered and there will be absolutely no risk of losing. But they did not realize that croupiers also have to earn some money. This is why the number zero was introduced into roulettes. Therefore, the total number of pockets is always odd and two bets are not enough to cover all of them (there would always be one pocket left).

To eliminate any risks, it was decided that two bets are not enough. In each game, three bets must be made instead of two, to cover all existing pockets. Your task is to compute the minimal price of such bets.



Input Specification

The input contains description of several roulettes. Each roulette description consists of two lines. The first line contains a single positive odd integer $N = 2 * K + 1$ ($3 \leq N < 200\,000$), the number of roulette pockets.

The second line contains N positive integers separated by a space, denoting the price of individual pockets, in the order they appear on the circumference of the wheel. The last pocket is supposed to neighbor with the first one. All prices will be between zero and 1 000.

The last roulette description is followed by a line containing zero.

Output Specification

For each roulette, output the price of the optimal set of bets. The set must have following properties:

1. It contains exactly three bets.
2. Each bet covers K neighboring pockets.
3. The bets together cover all roulette pockets (of course, some numbers may be covered twice).
4. The sum of prices of the bets is minimal among all such possible sets.

The price of a bet is the sum of prices for individual pockets.

Sample Input

```
5
1 2 3 4 5
9
1 2 3 4 5 6 7 8 9
0
```

Output for Sample Input

```
16
51
```

Dept. of Computer Science and Engineering, Czech Technical University in Prague
Faculty of Mathematics and Physics, Charles University in Prague
Faculty of Electrical Engineering and Computer Science, Technical University of Ostrava
Faculty of Informatics and Information Technologies, Slovak University of Technology
Faculty of Informatics, Masaryk University
Faculty of Management Science and Informatics, University of Zilina

CTU Open Contest 2008

Government Help

`help.c`, `help.C`, `help.java`, `help.p`

Two major banks in our country are knee-deep in troubles. We will call them "Bank A" and "Bank B" — hopefully you understand we absolutely *cannot* tell you their real names. It might ruin them.

As a logical consequence of their troubles, government comes to rescue them, preparing a set of financial "packages" that will be distributed between these two banks. However, there is one important condition: the packages will be given to the banks one by one and we need to minimize the difference between the amount given to both banks in any particular moment. Otherwise, one of the banks would gain a big advantage over the other.

For example, imagine that we have four packages with the values of 100 000, 110 000, 120 000, and 150 000. If we give the first package to the Bank A, then the second one to the Bank B, the third one to the Bank B, and the fourth package to the Bank A, the difference between the help provided to the banks will be +100 000 after the first package, -10 000 after the second one, -130 000 after the third one, and +20 000 after the last package. The maximal difference is therefore 130 000.

There is a better solution, with the maximal difference of 100 000 only: first, give 100 000 to the Bank A, then 120 000 to the Bank B, then 110 000 to the Bank A, and then 150 000 to the Bank B.

Input Specification

The input contains multiple sets of financial packages. Each set begins with a line containing a single positive integer N: the number of financial packages ($1 \leq N \leq 50\,000$). The second line is a set of N positive integers separated by a space, all of them between 100 000 and 199 999 inclusive. These numbers specify the amount of money in individual packages. Packages can be assigned in any order, the order of numbers on the second line is therefore not important.

The last set will be followed by a line containing single zero.

Output Specification

For each set of financial packages, output one line containing the order of assignments of packages to the banks, such as the maximal difference is as low as possible. If there are more possible solutions, you may print any of them.

Assignments must be separated by a space, each assignment consists of one number (the package size), one dash character ("`-`"), and an uppercase letter ("`A`" or "`B`") specifying the bank.

Sample Input

```
4
100000 110000 120000 150000
3
100000 100000 100000
0
```

Output for Sample Input

```
100000-A 120000-B 110000-A 150000-B
100000-B 100000-A 100000-A
```

Dept. of Computer Science and Engineering, Czech Technical University in Prague
Faculty of Mathematics and Physics, Charles University in Prague
Faculty of Electrical Engineering and Computer Science, Technical University of Ostrava
Faculty of Informatics and Information Technologies, Slovak University of Technology
Faculty of Informatics, Masaryk University
Faculty of Management Science and Informatics, University of Zilina

CTU Open Contest 2008

Tree Insertions

`insert.c`, `insert.C`, `insert.java`, `insert.p`

All modern banks employ information systems to process their data. The amount of data is enormous. Imagine all the transactions, payments, e-banking, web services, etc. Therefore, advanced data structures must be used to store the data and allow to access them very quickly.

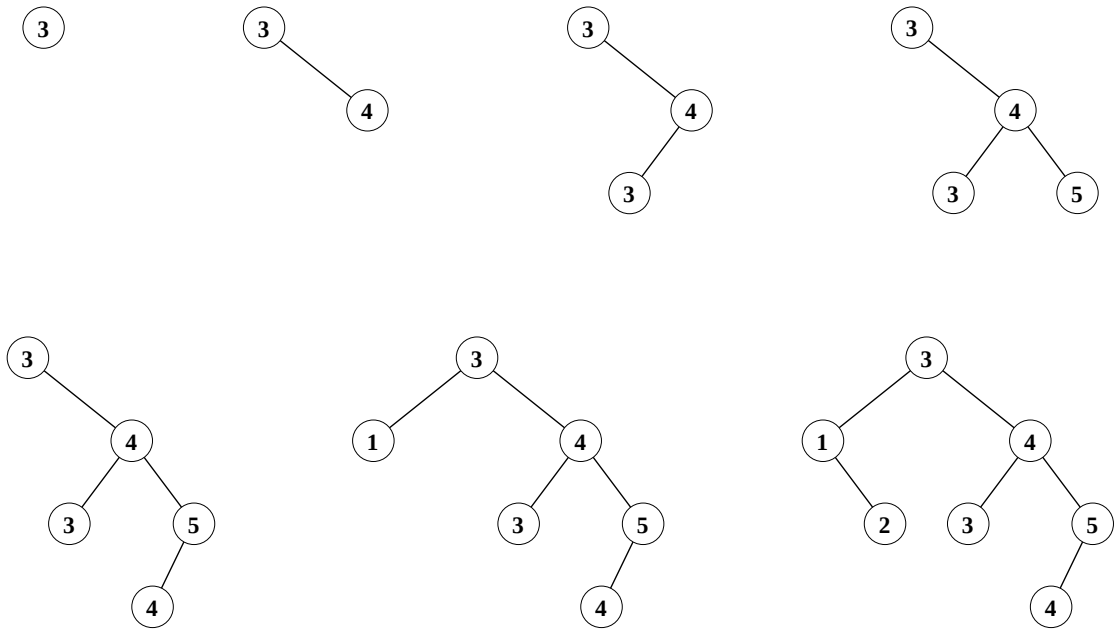
A *Binary Search Tree* (BST) is one example of such a data structure It holds a collection of values with some comparison operation that provides linear ordering on these values.

BST consists of nodes, each of them contains one value and has at most two subnodes: left and right (BST is a binary tree). The left subtree always contains only values strictly less than the node value, the right subtree only values greater than or equal to the node value.

As a consequence, values may easily be looked up by traversing the tree recursively. We begin with the root node and compare its value with the value we are searching for. Depending on the result, we descend either into the left or into the right subtree, but we never need to walk through both.

If we want to insert values to an existing tree, the procedure is also simple. The first value (when the tree is empty) is always put as the root. If the tree already exists, we start with the root node and traverse the tree recursively, as in the case of searching. When the traversal leads us to a missing subnode, we create a new leaf node at that position and assign it the new value.

The figures below show the tree after subsequently adding the following sequence of numbers: 3, 4, 3, 5, 4, 1, and 2.



You may notice that different permutations of the same numbers will often result in the same BST. For example, the tree from the fifth figure above may be constructed by three different input sequences:

- 3, 4, 3, 5, 4
- 3, 4, 5, 4, 3
- 3, 4, 5, 3, 4

Interesting, isn't it? Your task is to compute how many different permutations are there that will result into the same BST.

Input Specification

The input will consist of several trees, each of them specified on two lines. The first line contains a single integer N ($1 \leq N \leq 100$), the number of values in the tree. The second line contains N values separated by a space. These values, if inserted in the given order, form a BST to be examined. All values will be between 0 and 1 000.

The last tree is followed by a line containing a single zero.

Output Specification

For each tree, output the total number of different permutations that would generate the same Binary Search Tree. As you may notice in the Sample Output, this number may exceed 2^{32}.

Sample Input

```
5
3 4 3 5 4
7
3 4 3 5 4 1 2
31
16 8 24 4 12 20 28 2 6 10 14 18 22 26 30 1 3 5 7 9 11 13 15 17 19 21 23 25 27 29 31
0
```

Output for Sample Input

```
3
45
7483682586183598080000
```